ENGINEERING & COLLABORATION MANUAL

คู่มือนักพัฒนาและการทำงานร่วมกันเป็นทีม (Developer & Team Collaboration Guide)

แนวทางสถาปัตยกรรมระบบ กระบวนการทำงานร่วมกัน การแบ่งโมดูลโค้ด การควบคุมสิทธิ์ RBAC และขั้นตอนการส่งมอบระบบ

ชื่อโปรเจกต์	**AI Agent Visual Workflow Builder (LGv3)**
Repository หลัก	**HS2JFW/AI-Agent-Project-LGv3**
ระบบ Production	**https://aibuilder-dev.addpay.in.th**
ระบบ Deployment Platform	**Coolify Cloud (Project: ai-builder)**
Tech Stack	**FastAPI + LangGraph + React 19 + Vite**
เวอร์ชันเอกสาร	**Release v3.0.0 (30 ตุลาคม 2026)**

1. ภาพรวมสถาปัตยกรรมและเทคโนโลยี (Architecture & Tech Stack)

ระบบ **AI Agent Visual Workflow Builder** ถูกออกแบบขึ้นเพื่อให้ผู้ใช้งานสามารถลากวางโหนด (Visual Node Canvas) เพื่อสร้างและสั่งประมวลผลกระบวนการทำงานอัตโนมัติของ AI Agent ขนาดใหญ่ เช่น การวิเคราะห์เอกสาร TOR, การทำ Thai Verbatim OCR, การประเมินเกณฑ์จัดซื้อจัดจ้าง และการประมวลผลข้อมูล Big Data ด้วย In-Process SQL

FastAPI	React 19	LangGraph	5 Roles
Python 3.11 Backend	React Flow Canvas	Multi-Agent Engine	RBAC Security Layer

เลเยอร์ระบบ (Layer)	เทคโนโลยีที่ใช้งาน (Technologies)	บทบาทหน้าที่ (Responsibility)
Frontend Client	React 19, TypeScript, React Flow, Tailwind CSS v4, Lucide React, TipTap, Vite 8	เรนเดอร์ Canvas ผังโหนด, ตัวจัดการผู้ใช้ RBAC, SSE Log Stream และ UI Inspector
API & Business Logic	FastAPI, Pydantic v2, Python-JOSE, Passlib (Bcrypt)	จัดการ REST Endpoints, สิทธิ์ RBAC, จัดคิวไฟล์ และสร้าง Token JWT
Agent & Execution	LangGraph, Polars, DuckDB, Async BackgroundTasks	แปลงกราฟผังงานเป็น LangGraph State Machine และรันประมวลผลแบบ Real-time
Database & Storage	SQLAlchemy (Async), aiosqlite / SQLite (รองรับ Postgres)	จัดเก็บข้อมูลผู้ใช้, Workflow Graph, ประวัติการรัน และ Auto-Migration
DevOps & Infra	Docker Compose, Coolify, Traefik Reverse Proxy	ระบบ Auto-deploy, จัดการ SSL/TLS และแยก Container เครือข่าย

2. การติดตั้งและตั้งค่าเครื่องสำหรับพัฒนา (Local Development Setup)

2.1 โปรแกรมที่จำเป็นต้องมีในเครื่อง (Prerequisites)

- **Git:** v2.40+ สำหรับการทำงานกับ Version Control
- **Node.js:** v20 LTS หรือ v22 LTS (ติดตั้งผ่าน `nvm` หรือ `fnm`)
- **Python:** v3.11.x (แนะนำ `uv` package manager เพื่อความเร็วสูงสุด)
- **Docker Desktop:** สำหรับจำลอง Container ในโหมดพัฒนา

2.2 ขั้นตอนการติดตั้งโปรเจกต์ (Step-by-step Setup)

```
# 1. โคลน Repository ลงเครื่อง
git clone https://github.com/HS2JFW/AI-Agent-Project-LGv3.git ai-agent-builder
cd ai-agent-builder

# 2. ติดตั้ง Backend FastAPI (Python 3.11)
cd backend_fastapi
uv venv                       # หรือ python -m venv .venv
uv sync                       # หรือ pip install -r requirements.txt

# 3. ติดตั้ง Frontend React Flow
cd ../frontend
npm install
```

2.3 การรันระบบในโหมดพัฒนา (Development Run)

- **รันผ่านสคริปต์อัตโนมัติ (แนะนำ):** ดับเบิลคลิกไฟล์ `start.bat` ในโฟลเดอร์หลัก ระบบจะเปิดทั้ง Backend และ Frontend พร้อมเปิดเบราว์เซอร์ให้อัตโนมัติ
- **รันแยก Terminal เพื่อดู Log:**
  - Backend: `cd backend_fastapi && uv run uvicorn app.main:app --port 8001 --reload` (Swagger UI: `http://localhost:8001/docs`)
  - Frontend: `cd frontend && npm run dev` (เปิดที่ `http://localhost:5173` หรือ `5176`)

3. โครงสร้างโค้ดและการแบ่งโมดูลเพื่อลดการชนกัน (Code Modularization)

> **ข้อพึงระวังสำคัญ:** ไฟล์ `frontend/src/App.tsx` มีขนาดใหญ่มาก (27,000+ บรรทัด) หาก Dev หลายคนแก้ไขในไฟล์นี้พร้อมกัน จะเกิด **Git Merge Conflict** ที่แก้ไขยากมาก ทางทีมจึงกำหนดระเบียบการแยกโค้ดดังนี้:

3.1 แนวทางการแยกชิ้นส่วน Frontend Components

```
frontend/src/
├── components/
│   ├── UserManagementModal.tsx # หน้าจอจัดการผู้ใช้ RBAC (แยกไฟล์เดี่ยว)
│   ├── UnifiedSettingsModal.tsx
│   ├── UserProfileModal.tsx
│   ├── canvas/                 # Component ย่อยบน Canvas
│   │   ├── FlowControls.tsx
│   │   └── TimerCountdown.tsx
│   └── dialogs/                # Floating Dialogs
├── hooks/                      # Logic & State Management
│   ├── useWorkflowState.ts
│   └── useExecution.ts
└── App.tsx                     # Main Canvas (ให้เรียกใช้เฉพาะ Component ย่อย)
```

3.2 กฎการเขียนโค้ดสำหรับนักพัฒนาทุกคน

1. **ห้ามเขียน Component ยาวๆ ลงใน `App.tsx` ตรงๆ:** ให้สร้างไฟล์แยกใน `frontend/src/components/` เสมอ แล้วค่อย Import เข้ามาใช้
2. **แยก Logic และ API Fetch ออกไปไว้ใน Custom Hooks:** จัดเก็บในโฟลเดอร์ `frontend/src/hooks/`
3. **คง TypeScript Type Safety:** ก่อนส่งงานทุกครั้ง ต้องมั่นใจว่ารัน `npx tsc -b` แล้วไม่มี Error

4. มาตรฐาน Git และกระบวนการเปิด Pull Request (Git Collaboration)

4.1 การแบ่ง Branch (Branching Model)

ชื่อ Branch	วัตถุประสงค์ (Purpose)	สิทธิ์และการส่งโค้ด (Deployment Trigger)
`main`	โค้ด Production ที่นิ่งและพร้อมให้บริการ	ห้าม Push ตรงเด็ดขาด! ต้องผ่าน PR จาก `develop` และทดสอบสมบูรณ์
`develop`	สาขาหลักสำหรับการรวมงานในรอบ Sprint	Deploy อัตโนมัติไปยัง Staging Environment สำหรับทดสอบระบบ
`feat/<name>`	สาขาสำหรับพัฒนาฟังก์ชันใหม่ของ Dev แต่ละคน	แตกออกมาจาก `develop` เมื่อเสร็จให้เปิด PR กลับเข้า `develop`
`fix/<name>`	สาขาสำหรับแก้ไขบักหรือปรับปรุงด่วน	แตกออกมาแก้ไข และเปิด PR ผ่านการ Review ตามลำดับ

4.2 มาตรฐานข้อความ Commit (Conventional Commits)

- `feat: <คำอธิบาย>` เช่น `feat: add export workflow to json button`
- `fix: <คำอธิบาย>` เช่น `fix: handle null owner_id in workflow response`

- `refactor: <คำอธิบาย>` เช่น `refactor: extract canvas toolbar from App.tsx`
- `test: <คำอธิบาย>` เช่น `test: add operator forbidden execution test case`
- `docs: <คำอธิบาย>` เช่น `docs: update api endpoints in README`

4.3 Checklist ก่อนการเปิด Pull Request (PR Verification Gate)

> ทุก Pull Request จะต้องผ่านการตรวจสอบ 3 ขั้นตอนด้านล่างนี้ก่อน จึงจะได้รับอนุมัติให้ Merge เข้าสู่สาขาหลัก:
>
> 1. **Backend Pytest ผ่าน 100%:** รันคำสั่ง `cd backend_fastapi && uv run pytest` (ต้องผ่านครบทุกชุด)
> 2. **Frontend Type Check & Build ผ่าน:** รันคำสั่ง `cd frontend && npx tsc -b && npm run build`
> 3. **Code Review:** ได้รับการตรวจสอบและกด Approve จาก Lead Developer อย่างน้อย 1 ท่าน

5. ระบบควบคุมสิทธิ์ RBAC สำหรับนักพัฒนา (Security Guidelines)

5.1 สิทธิ์ 5 ระดับ (5-Tier Role System)

Role	Tier	ความสามารถในระบบ (Allowed Capabilities)	ข้อห้าม (Restricted)
SUPERADMIN	Tier 1	จัดการบัญชีผู้ใช้, แก้ไขและลบ Workflow ได้ทุกตัว, ดู System Logs	ไม่มี
ADMIN	Tier 2	เข้าเมนู User Management, ดูแล Workflow ในภาพรวมทั้งหมด	ห้ามยกระดับตนเองเป็น Superadmin
BUILDER	Tier 3	สร้างและแก้ไข Workflow ของตนเอง, รันและทดสอบโหนดใน Flow	ห้ามแก้หรือลบ Flow ของคนอื่น, ไม่เห็นเมนูจัดการผู้ใช้
OPERATOR	Tier 4	ส่งไฟล์เอกสารเข้าคิว, สั่งรัน Workflow, ดาวน์โหลดผลลัพธ์	ห้ามสร้าง, แก้ไข หรือลบโครงสร้าง Node ใน Flow
VIEWER	Tier 5	ดูผัง Canvas และสถิติการรันงานแบบ Read-only	ห้ามสั่งรัน, ห้ามบันทึก และห้ามแก้ไขใดๆ ทั้งสิ้น

5.2 การใช้งาน Guard Dependency ใน Backend FastAPI

```
from fastapi import APIRouter, Depends
from app.routers.auth import require_role
from app.models.user import User

# ตัวอย่างการจำกัดสิทธิ์เฉพาะ Admin และ Superadmin
@router.post("/api/admin/system-reset")
async def system_reset(current_user: User = Depends(require_role("superadmin", "admin"))):
    return {"status": "ok", "admin": current_user.username}

# ตัวอย่างการสร้าง Workflow โดยผูก owner_id เข้ากับผู้ใช้ที่ล็อกอิน
@router.post("/api/workflows")
async def create_flow(payload: WorkflowCreate, current_user: User = Depends(require_role("superadmin", "ad
    new_wf = Workflow(name=payload.name, owner_id=current_user.id)
    ...
```

6. การทดสอบระบบและการประกันคุณภาพ (QA & Verification)

6.1 บัญชีสำหรับการทดสอบจำลองสิทธิ์ (Test User Accounts)

Username	Password	Role	วัตถุประสงค์ในการทดสอบ (Test Objective)
user01	`admin`	SUPERADMIN	ทดสอบฟังก์ชันจัดการผู้ใช้, สิทธิ์เข้าถึงทุกหน้าจอ และการลบ Flow
admin	`admin123`	ADMIN	ทดสอบการเข้าถึง User Management และการบริหารจัดการระบบ
builder01	`builder123`	BUILDER	ทดสอบการสร้าง/แก้ Flow ของตนเอง และทดสอบว่าแก้ Flow คนอื่นไม่ได้
operator01	`operator123`	OPERATOR	ทดสอบการอัปโหลดไฟล์, กดสั่งรัน Flow และทดสอบว่าถูกบล็อกไม่ให้แก้ Flow

Username	Password	Role	วัตถุประสงค์ในการทดสอบ (Test Objective)
viewer01	`viewer123`	VIEWER	ทดสอบโหมด Read-only ว่าถูกบล็อกไม่ให้สั่งรันและไม่ให้บันทึกผัง Flow

6.2 คำสั่งรันชุดทดสอบอัตโนมัติ (Automated Test Commands)

```
# รัน Pytest ครบทุกชุด (Analytics, Auth, LangGraph, RBAC, Workflows)
cd backend_fastapi
uv run pytest

# รันเฉพาะชุดทดสอบ RBAC สิทธิ์ผู้ใช้งาน
uv run pytest tests/test_rbac.py tests/test_workflow_rbac.py -v
```

7. การส่งขึ้นเซิร์ฟเวอร์และการทำงานกับ Coolify (Deployment Pipeline)

7.1 กฎระเบียบด้านพอร์ตและ Reverse Proxy สำหรับ Developer

- **หลีกเลี่ยงการเปิด Host Port:** ในไฟล์ `docker-compose.yml` **ห้าม** ใช้คำสั่ง `ports: ["80:80"]` เนื่องจากพอร์ต 80 ของเซิร์ฟเวอร์ถูกสงวนไว้ให้ Traefik Proxy ประจำตัวของ Coolify
- **ใช้ Expose สำหรับ Container Networking:** ให้ใช้คำสั่ง `expose: ["80"]` สำหรับบริการ Frontend เสมอ
- **การตั้งค่า Domain บน Coolify:** ต้องกำหนดผ่านฟิลด์ `docker_compose_domains` ในแอปพลิเคชัน Coolify เพื่อให้ Traefik สร้าง SSL Certificate อัตโนมัติ

7.2 สรุปลิงก์สำคัญประจำระบบ (System Directory & URLs)

บริการ / เครื่องมือ	ลิงก์ URL	คำอธิบาย
Web Application (Canvas)	https://aibuilder-dev.addpay.in.th	หน้าจอ Visual Workflow Canvas และระบบหลัก
User Management (Direct Link)	https://aibuilder-dev.addpay.in.th/?modal=users	ลิงก์ตรงเปิดหน้าต่างจัดการผู้ใช้ RBAC ทันที
Interactive API Documentation	https://aibuilder-dev.addpay.in.th/docs	Swagger UI สำหรับทดสอบ REST APIs ทุกตัว
Backend Health Check	https://aibuilder-dev.addpay.in.th/health	Endpoint ตรวจสอบสถานะ FastAPI Server
Coolify Server Management	https://coolify.addpay.in.th	แผงควบคุมระบบ (โครงการ: `ai-builder`)

> **ข้อความส่งท้ายสำหรับทีมพัฒนา:** การร่วมกันพัฒนาโปรเจกต์นี้ให้ยึดหลัก Clean Code, มี Unit Test กำกับทุกฟังก์ชันใหม่ และแยกการพัฒนาผ่าน Branch เสมอ เพื่อให้ระบบมีความเสถียรและสามารถขยายต่อได้อย่างยั่งยืนครับ